



UNIVERSITÀ
DI TRENTO

Department of
Information Engineering and Computer Science

Automated Reasoning and **Formal Verification**

Laboratory 12

Gabriele Masina

gabriele.masina@unitn.it

<https://github.com/masinag/arfv2026>

Università di Trento

May 27, 2026

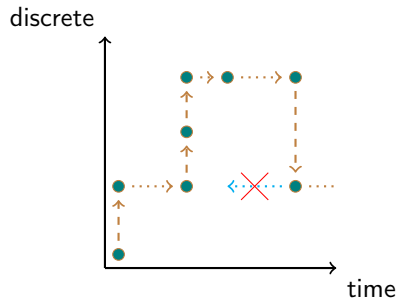
These slides are derived from those by Stefano Tonetta, Alberto Griggio, Silvia Tomasi, Thi Thieu Hoa Le, Alessandra Giordani, Patrick Trentin, Giuseppe Spallitta for FM lab 2005-2024.

Hybrid Systems

A dynamical system that exhibits both **continuous** and **discrete** dynamic behavior:

- ▶ **flow** (described by differential equations)
- ▶ **jump** (described by a state machine or automaton).

- ▶ In general **model checking on hybrid systems is undecidable**.
- ▶ Many sub-classes:
 - ▶ *rectangular* (decidable)
 - ▶ *linear* (undecidable)



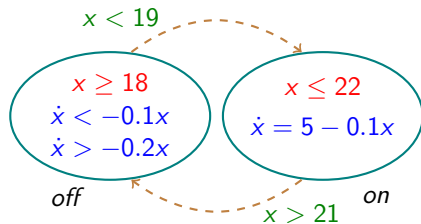
Hybrid Automata

Explicit graph representation of discrete states (**nodes**) and transitions (**edges**) plus symbolic description of the continuous dynamics:

- ▶ **location invariants**
- ▶ **transition guards**
- ▶ **flow**: derivative w.r.t time (polytopes)

Note

Continuous dynamics must be described by convex polytopes (i.e., conjunctions of linear inequalities).





Outline

1. HYCOMP

2. Example

3. Homework



- ▶ Supports the modelling and verification of a network of hybrid automata
- ▶ Supports invariant and LTL properties
- ▶ It encodes the Hybrid model into a “standard” nuXmv model

Docs and details: http://www.sergiomover.eu/paper/mover_phd_thesis.pdf

Download: <https://hycomp.fbk.eu/download.html>

The input language is called HYDI (Hybrid automata with Discrete interaction). It extends the nuXmv input language with

events: introduced with the keyword **EVENT**. Intuitively, transitions are distinguished by the event which is being fired. They are used to **synchronize** different modules.

continuous variables: declared with the keyword **continuous**. Their value can change in a timed transition and **evolve** according to some function continuous in time.

flow conditions: used to **constrain the continuous evolution** of continuous variables; the constraints are introduced with the keyword **FLOW** and may refer to the derivatives of continuous variables, denoted with **der**

HyCOMP updates

- ▶ **TRANS** constrain the discrete behaviour only
- ▶ **INVAR**: **continuous** allowed in invariants with shape:
`no_continuous_expr -> convex_continuous_expr`

Read and Compile Model

- ▶ `hycomp_read_model`
- ▶ `hycomp_compile_model`
- ▶ `hycomp_untime_network -d`
- ▶ `hycomp_async2sync_network`
- ▶ `hycomp_net2mono`

Check Properties

- ▶ `hycomp_check_invar_*`
- ▶ `hycomp_check_ltl*`



Outline

1. HYCOMP

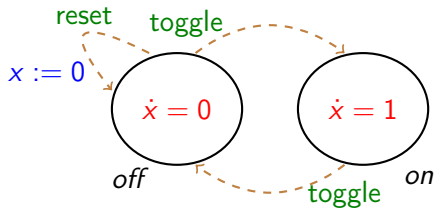
2. Example

3. Homework

Example: Stopwatch [1/3]

Example: Stopwatch

- ▶ Write a HYDI model that represents the hybrid automaton in the picture.
- ▶ Add an asynchronous process that controls the stop-watch using the **toggle** and **reset** commands.
- ▶ Check if the following properties hold:
 - ▶ x is always non-negative
 - ▶ if the stop-watch is on state **off**, then x is always zero



MODULE Stopwatch

VAR

mode : {on, off};

x : continuous;

EVENT toggle, reset;

FLOW mode = on -> der(x) = 1;

FLOW mode = off -> der(x) = 0;

TRANS EVENT = reset -> next(x) = 0;

TRANS EVENT != reset -> next(x) = x;

TRANS EVENT = toggle -> next(mode) != mode;



Example: Stopwatch [3/3]

```
MODULE Controller  
EVENT toggle, reset;
```

```
MODULE main  
VAR
```

```
    stopWatch : Stopwatch;  
    controller : Controller;
```

```
SYNC controller, stopWatch EVENTS toggle, toggle;  
SYNC controller, stopWatch EVENTS reset, reset;
```



Outline

1. HYCOMP

2. Example

3. Homework

Bouncing ball

- ▶ A ball is initially at height 10.
- ▶ We let the ball fall and bounce.
- ▶ Every time the ball bounces half its speed is lost.
- ▶ The gravitational acceleration is 9.8.